

طراحی و بهینه‌سازی کنترلر PID با استفاده از الگوریتم جستجوی گرانشی

وحید محمدنژاد، مریم ذکری

دانشگاه صنعتی اصفهان، دانشکده برق و کامپیوتر، hivahid@yahoo.com

دانشگاه صنعتی اصفهان، دانشکده برق و کامپیوتر، mzekri@cc.iut.ac.ir

چکیده - در این مقاله به بررسی الگوریتم تکاملی و ایرانی جستجوی گرانشی (2009) در طراحی و بهینه‌سازی کنترلر PID می‌پردازیم. این کنترلرها به خاطر کارایی خوب در طیف وسیعی از پروسه‌های صنعتی و تحلیل آسان عملکردشان به خاطر ساختار ساده‌شان چندین دهه است که مورد محبوبیت قرار گرفته‌اند. به خاطر کاربردی بودن نتایج آزمایشات و از آنجا که یکی از موضوعات مهم در سیستم‌های الکتریکی خصوصاً قدرت، تامین ولتاژ خروجی ثابت می‌باشد؛ سیستم نمونه جهت آزمایش الگوریتم جستجوی گرانشی، سیستم کنترل ولتاژ خروجی یک ژنراتور سنکرون انتخاب شده است. در این راستا از این الگوریتم در طراحی کنترلر PID برای کنترل ولتاژ خروجی این ژنراتور سنکرون بهره برده و نتایج حاصل را با نتایج حاصل از الگوریتم ژنتیک و روش زیگلر-نیکولز مقایسه گردیده است. هر چند الگوریتم جستجوی گرانشی حافظه‌دار نمی‌باشد اما نتایج نشان‌دهنده برتری الگوریتم جستجوی گرانشی نسبت به دو روش دیگر به خصوص الگوریتم حافظه‌دار ژنتیک در یافتن کنترلر بهینه PID برای سیستم مذکور می‌باشد.

کلید واژه- الگوریتم جستجوی گرانشی (GSA)، سیستم‌های اتوماتیک رگولاتور ولتاژ (AVR)، طراحی کنترلر PID

1-مقدمه

امروزه با افزایش ابعاد مسایل و اهمیت یافتن سریع پاسخ در بسیاری از مسایل، الگوریتم‌های ابتکاری¹ همچون الگوریتم ژنتیک² GA (1975)، الگوریتم جمعیت مورچگان³ (1991) و ... جایگزین روش‌های کلاسیک شده‌اند. یک نوع از این مسایل، مسایل بهینه‌سازی است؛ که در مهندسی یکی از موضوعات مهم می‌باشد. الگوریتم جستجوی گرانشی GSA⁴ نیز یک الگوریتم ابتکاری است که در سال 2009 مطرح گردیده است. در این مقاله با استفاده از این الگوریتم به طراحی و بهینه‌سازی کنترلر PID⁵ برای یک سیستم نمونه می‌پردازیم. جهت کاربردی بودن نتیجه شبیه‌سازی‌ها، سیستم نمونه را با توجه به یک سیستم واقعی مدل‌سازی کرده‌ایم. این سیستم که مربوط به کنترل ولتاژ خروجی⁶ یک ژنراتور سنکرون می‌باشد؛ با توجه به

مقاله ژورنال 2009 [2] انتخاب شده است. در ادامه بعد از معرفی کنترلر PID و الگوریتم جستجوی گرانشی، به کمک نرم-افزار Matlab 7.10 به شبیه‌سازی سیستم فوق پرداخته و نتایج حاصل از الگوریتم جستجوی گرانشی را با نتایج حاصل از الگوریتم ژنتیک و روش زیگلر-نیکولز حلقه بسته مقایسه خواهیم کرد.

2-کنترلر PID

در این کنترل‌کننده‌ها عموماً فرمان خروجی براساس خطای بین متغیر کنترل‌شده و مقدار مرجع صادر می‌گردد. کنترلرهای PID به خاطر کارایی خوب در طیف وسیعی از پروسه‌های صنعتی و سادگی ساختار و در نتیجه تحلیل آسان عملکردشان از سال 1940 که پروسه‌های کنترلی پدیدار شدند؛ به عنوان کنترل‌کننده‌ای استاندارد شناخته می‌شدند. یکی از فرم‌های متداول برای نمایش تابع تبدیل این کنترلر در حوزه فرکانس به صورت رابطه (1) می‌باشد.

$$U(s) = K \times \left(1 + \frac{1}{T_i s} + T_d s\right) \times E(s) \quad (1)$$

که $E(s)$ سیگنال خطای ورودی کنترلر و $U(s)$ سیگنال خروجی کنترلر می‌باشد. K را بهره تناسبی، T_i را زمان

¹ -Heuristic.

² -Genetic algorithm.

³ -Ant colony search algorithm.

⁴ - Gravitational Search Algorithm.

⁵ - Proportional-Integral-Derivative controller

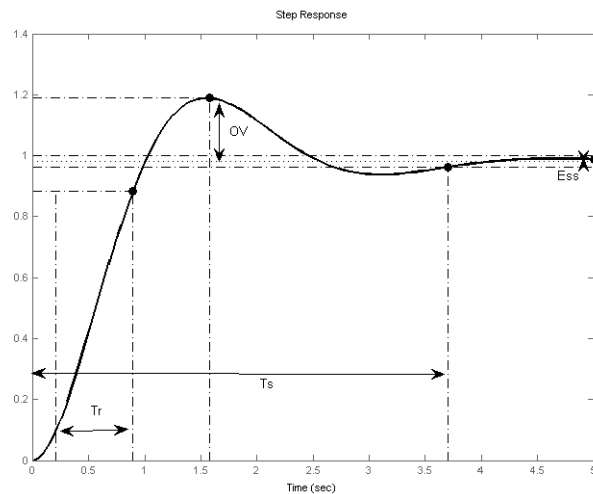
⁶ - Automatic Voltage Regulator (AVR)

انتگرالگیر و T_d را زمان مشتگیر می‌نامند. رابطه 1 را می‌توان به فرم ساده‌تر زیر نوشت.

$$U(s) = (K_p + \frac{K_i}{s} + K_d s) \times E(s) \quad (2)$$

که $K_p = K$ ، $K_i = K/T_i$ و $K_d = K T_d$. در ادامه ما از فرم دوم کنترلر استفاده خواهیم کرد.

معمولاً معیار بررسی کارایی کنترلر براساس مشخصات پاسخ سیستم به یک سیگنال تست تعیین می‌گردد. از مشخصات متداول پاسخ یک سیستم در حوزه زمان می‌توان به میزان فراجش¹، زمان صعود²، زمان نشست³ و خطای حالت دایم⁴ اشاره کرد (شکل 1).



شکل 1- نمایش میزان فراجش OV، زمان صعود t_r ، زمان نشست t_s و خطای حالت دایم Ess

از معیارهای دیگر کنترلر میزان خطایی است که پاسخ سیستم از مقدار مرجع دارد. در این زمینه معیارهای متفاوتی چون IAE^5 ، $ITAE^6$ ، ... مطرح شده است. از این میان در ادامه مقاله از معیار ITAE استفاده خواهیم کرد.

$$ITAE = \int_0^{\infty} t|e(t)| dt \quad (3)$$

3- الگوریتم جستجوی گرانشی GSA

3-1- الگوریتم جستجوی گرانشی.

الگوریتم جستجوی گرانشی را می‌توان به عنوان یک سیستم ایزوله شده از اجسام در نظر گرفت که شبیه یک فضای مصنوعی کوچک از اجسام است و قوانینی شبیه قانون گرانش و قوانین

حرکت نیوتن در محیطی با زمان گسسته، بر آن حاکم می‌باشد. در این فضا هر جسم نماینده‌ی یک جواب است که با گذشت زمان انتظار داریم اجسام توسط جسم سنگینتر جذب شوند. این جسم بیان‌کننده پاسخ بهینه در فضای جستجو است. در ادامه گام به گام به تشریح این الگوریتم می‌پردازیم.

گام 1- تشکیل سیستم: در قدم اول فضای سیستم مشخص می‌شود. محیط شامل یک دستگاه مختصات چند بعدی در فضای تعریف مسئله است. هر نقطه از فضا جواب مسئله است و عامل‌های جستجوکننده‌ی مجموعه‌ای از اجسام هستند. هر جسم دو مشخصه مهم جرم و موقعیت دارد. جرم هر جسم براساس تابع هدف تعیین می‌گردد و موقعیت جسم نیز، نقطه‌ای در فضا است که جوابی از مسئله می‌باشد. در یک فضای n بعدی موقعیت هر جسم به صورت زیر مشخص می‌شود.

$$X_i = (X_i^1, X_i^2, \dots, X_i^n) \quad (4)$$

که X_i^n موقعیت بعد n ام از جسم i ام می‌باشد.

گام 2- وضع قوانین:

1- قانون گرانش: هر جسم در سیستم مصنوعی تمام اجسام دیگر را با نیروی جاذبه به سمت خود دعوت می‌کند. در زمان مشخص t ، نیروی جاذبه اعمالی روی جسم i از طرف جسم j در راستای بعد d بصورت زیر محاسبه می‌شود.

$$F_{ij}^d = G(t) \times \frac{M_j(t) \times M_i(t)}{R_{ij} + \epsilon} (X_j^d(t) - X_i^d(t)) \quad (5)$$

که R_{ij} فاصله اقلیدسی دو جسم، $G(t)$ ثابت گرانش⁷ و ϵ نیز یک عدد بسیار کوچک می‌باشد. برای این که الگوریتم به صورت تصادفی عمل کند، فرض می‌کنیم نیروی مجموع که بر جرم i در بعد d وارد می‌شود برابر با مجموع ضریبهای تصادفی از تمام نیروهایی که سایر اجسام بر جسم وارد می‌کنند (رابطه 6).

$$F_i^d(t) = \sum_{j=1, j \neq i}^n \text{rand}_j \times F_{ij}^d(t) \quad (6)$$

در این رابطه rand_j یک عدد تصادفی با توزیع یکنواخت در بازه $[0,1]$ است

2- قانون حرکت: سرعت فعلی هر جسم برابر با مجموع ضریبی از سرعت قبلی جسم و تغییر سرعت آن است. تغییر سرعت یا شتاب هر جسم نیز در بعد d برابر با نیروی وارد بر آن جسم در جهت بعد d تقسیم بر جرم جسم می‌باشد (رابطه 7)

$$a_i^d(t) = \frac{F_i^d(t)}{M_i(t)} \quad (7)$$

شتاب جسم i در جهت بعد d در زمان t با $a_i^d(t)$ و جرم عامل i با M_i نشان داده می‌شود. حال چنانچه روابط فوق را در کنار یکدیگر قرار دهیم رابطه زیر برای شتاب به دست می‌آید.

¹ - Over shoot

² - Rise Time

³ - Settling Time

⁴ - Steady State Error

⁵ - integral absolute error

⁶ - integral of time-weighted absolute error

⁷ - Gravitational constant

$$a_i^d = G(t) \sum_{j=1, j \neq i}^N \left[\text{rand}_j \frac{M_j(t)}{R_{ij}(t) + \epsilon} (X_j^d - X_i^d(t)) \right] \quad (8)$$

سرعت بعدی هر جرم برابر مجموع ضربی از سرعت فعلی جرم و شتاب جرم است. بنابراین موقعیت و سرعت جرم را می توان بصورت زیر محاسبه کرد.

$$V_i^d(t+1) = \text{rand}_i \times V_i^d(t) + a_i^d(t) \quad (9)$$

$$X_i^d(t+1) = X_i^d(t) + V_i^d(t+1) \quad (10)$$

در این روابط $V_i^d(t)$ سرعت بعد d م جسم i در زمان t و rand_i یک عدد تصادفی در بازه $[0,1]$ است.

گام 3- تنظیم جرم اجسام: تنظیم جرم اجسام بر مبنای تابع هدف آنها انجام می شود. به گونه ای که به اجسام با شایستگی بیشتر، جرم بیشتر نسبت داده می شود. این بدین معنا است که جسم بهتر دارای قدرت جذب بیشتر و حرکت کندتر است. این کار با استفاده از رابطه 11 میسر می شود. در نهایت جرم عامل ها طبق رابطه 12 نرمالیزه می شود.

$$M_i(t) = \frac{\text{Fit}_i(t) - \text{Worst}(t)}{\text{best}(t) - \text{Worst}(t)} \quad (11)$$

$$M_i(t) = \frac{M_i(t)}{\sum_{j=1}^n M_j(t)} \quad (12)$$

که $\text{Fit}_i(t)$ بیانگر میزان برازندگی جرم i در زمان t است. $\text{best}(t)$ و $\text{Worst}(t)$ به ترتیب بیانگر میزان شایستگی ضعیفترین و قویترین عامل جمعیت در زمان t هستند که مقادیر آنها در مسائل کمینه یابی به ترتیب با استفاده از روابط 13 و 14 قابل محاسبه هستند.

$$\text{best}(t) = \min(\text{fit}_i(t)) \quad i \in \{1, 2, \dots, N\} \quad (13)$$

$$\text{Worst}(t) = \max(\text{fit}_i(t)) \quad i \in \{1, 2, \dots, N\} \quad (14)$$

در مسائل بیشینه یابی جای روابط فوق برای محاسبه بهترین بدترین مقدار شایستگی عوض می شود.

گام 4- تنظیم ثابت گرانش: ثابت گرانش، یک پارامتر مناسب برای کنترل توانایی های کاوش و بهره وری در این الگوریتم به شمار می آید. مقادیر بزرگ برای این پارامتر باعث تقویت توانایی کاوش الگوریتم و مقادیر کوچک آن باعث افزایش توانایی بهره وری الگوریتم می شود. طبق آزمایشات متعدد انجام شده توسط ارائه دهنده گان این الگوریتم استفاده از رابطه نمایی (رابطه 15) جهت کاهش ثابت گرانش در حل بسیاری از مسائل موثر است.

$$G(t) = g \times e^{\frac{-t}{T}} \quad (15)$$

که در آن g و T ضرایب مثبت ثابت و T بیانگر طول عمر سیستم است.

گام 5- گذر زمان حرکت اجسام و بروز رسانی پارامترها: در ابتدای تشکیل سیستم هر جسم به صورت تصادفی در یک نقطه از فضا قرار می گیرد که جوابی از مسئله است. در هر بار تکرار

الگوریتم، روابط 8 الی 10 محاسبه شده و جرم به موقعیت جدید تغییر مکان می دهد. پارامترهای سیستم شامل جرم جسم و ثابت گرانش نیوتن می باشند که در هر مرحله طبق روابط 11 تا 15 بروز رسانی می شوند. شرط توقف می تواند پس از مدت زمان مشخصی و یا تعداد مشخصی از تکرار الگوریتم تعیین شود. شبه کد این الگوریتم به صورت زیر می باشد:

1- تعیین محیط سیستم و مقدار دهی اولیه.

2- جابه جایی اولیه اجسام.

3- ارزیابی اجسام.

4- به روز رسانی پارامترهای G , $\text{best}(t)$, $\text{worst}(t)$, $M(t)$.

5- محاسبه نیروی وارد بر هر جسم.

6- محاسبه شتاب و سرعت هر جسم.

7- به روز رسانی موقعیت اجسام.

8- در صورتی که شرط توقف برآورده نشده به مرحله 3 برو. در غیر این صورت، بهترین جواب دیده شده تاکنون را در خروجی چاپ کن و الگوریتم را متوقف کن.

2-3 الگوریتم جستجو گرانشی نخبه گرا EGO^1

با توجه به رابطه 8 برای کاهش حجم محاسباتی الگوریتم می توان تعداد حضور اجسام را در این رابطه کاهش داد. برای این منظور می توان پیشنهاد کرد که تعداد محدودی (k جسم) از اجسام که شایستگی بیشتری دارند یا به عبارتی از جرم بزرگتری برخوردارند در تعیین شتاب سایر اجسام تاثیر داشته باشند و از مشارکت سایر اجسام با شایستگی کمتر در تعیین اندازه حرکت سایر اعضای جمعیت جلوگیری شود. البته در اجرای این پیشنهاد باید بسیار مراقب بود، زیرا که با اعمال چنین سیاستی توان کاوش الگوریتم کاهش و توان بهره وری آن افزایش می یابد. برای تحقق این هدف مقدار k به صورت متغیر با زمان تعریف می شود. به این صورت که در زمان شروع جستجوی تمام اجسام روی یکدیگر تاثیر می گذارند و با گذشت زمان از تعداد اعضای تاثیر گذار بر جمعیت به کاسته می شود. روش فوق از بار محاسباتی الگوریتم به مقدار قابل توجهی کم می کند (رابطه 16).

$$F_i^d = \sum_{j=k\text{best}(t), j \neq i}^N \text{rand}_j \times F_{ij}^d(t) \quad (16)$$

در این رابطه $k\text{best}(t)$ بیانگر مجموعه k جسم برتر جمعیت است. متناسب با این تغییر رابطه 8 نیز به صورت زیر تغییر می کند.

$$a_i^d = G(t) \sum_{j=k\text{best}(t), j \neq i}^N \left[\text{rand}_j \frac{M_j(t)}{R_{ij}(t) + \epsilon} (X_j^d - X_i^d(t)) \right] \quad (17)$$

3-3 الگوریتم جستجوی گرانشی اصلاح شده.

پارامتر با مقادیر حاصل از مراحل قبلی می‌باشد. برای پیاده سازی الگوریتم ژنتیک از جعبه ابزار این الگوریتم در matlab2010 تحت نام optimtool (در نسخه های پایین تر تحت نام gatool) استفاده شده است. جهت پیاده سازی الگوریتم جستجوی گرانشی جعبه ابزاری مشابه optimtool تحت نام GSA طراحی گردیده است. تنظیمات دو الگوریتم به صورت جدول 1 می باشد.

جدول 1- تنظیمات دو الگوریتم GSA و GA

GSA	GA	
50	50	تعداد عاملها
50	50	تعداد تکرار الگوریتم
%10	%10	در صد جهش
[0, 1]	[0, 1]	محدوده متغیرها
$k_{best}(t) = 5 + 45 \times e^{-t/5}$ $G(t) = 0.05 \times e^{-t/5}$	سایر تنظیمات همان پیش فرض جعبه ابزار gatool می-باشد	-

نتایج شبیه سازی را می‌توانید در انتها مقاله در جدول 2 و شکل‌های 3 الی 7 مشاهده کنید.

5-نتیجه گیری

نتایجی که می‌توان با توجه به شبیه‌سازی انجام گرفته برداشت کرد به شرح زیر می‌باشد.
1-همانطور که از جدول 2 و شکل 3 پیداست الگوریتم جستجوی گرانشی بهتر از دیگر روشها مقدار بهینه (مینیمم) را برای تابع هدف انتخاب شده یافته است.
2-از شکل‌های 6 و 7 نیز معلوم است الگوریتم جستجوی گرانشی سریعتر از الگوریتم ژنتیک به مقدار بهینه می‌رسد.
3-از شکل‌های 6 و 7 مشخص می‌شود که الگوریتم جستجوی گرانشی به خاطر جابه‌جایی تمام عاملها در هر تکرار، جواب بهینه یافته شده را تقریباً در هر تکرار بهینه‌تر می‌کند ولی در الگوریتم ژنتیک ممکن است در چندین مرحله جواب بهینه تغییر نکند.

الگوریتم جستجوی گرانشی بر خلاف بسیاری از الگوریتم‌های دیگر حافظه ندارد؛ اما عامل‌های جستجو کننده پاسخ در این الگوریتم بعد از یافتن پاسخ باز هم به حرکت خود ادامه می‌دهند هر چند این جابه‌جایی کوچک است اما برای اطمینان از این که بهترین جواب را از دست ندهیم می‌توانیم به مشابهت الگوریتم‌های حافظه دار بهترین جواب هر مرحله را ذخیره کرد. اگر بهترین جواب مرحله قبلی از مرحله کنونی بهتر باشد آنگاه بهترین جواب مرحله قبلی را با بدترین جواب مرحله کنونی جایگزین کنیم.

یکی از عیبهای الگوریتم جستجوی گرانشی وجود فقط نیروی جاذبه است که احتمال به دام افتادن الگوریتم در بهینه های محلی را افزایش می‌دهد. به مشابهت الگوریتم ژنتیک می‌توان درصدی از ضعیفترین اجسام را در هر مرحله حذف کرده و به جای آن اجسامی با یک موقعیت تصادفی به جای آنها جایگزین کرد. این کار که مشابه عمل جهش در الگوریتم ژنتیک می‌باشد باعث افزایش توانایی این الگوریتم در یافتن بهینه های فرا محلی گردیده است.

4-شبیه‌سازی

در این شبیه سازی به مقایسه توانایی الگوریتم‌های جستجوی گرانشی، ژنتیک و روش زیگلر-نیکولز در یافتن یک کنترلر PID بهینه برای داشتن یک پاسخ پله ایده‌آل برای یک سیستم نوعی می‌پردازیم. در این سیستم هدف کنترل ولتاژ خروجی یک ژنراتور سنکرون می‌باشد (شکل 2).

تابع هدف به صورت $w(K) = F \times ITAE$ انتخاب شده است که

$$F = (1 - e^{-\beta})(0v + e_{ss}) + e^{-\beta}(t_s - t_r) \quad (18)$$

$$ITAE = \text{Avrege}(\sum_{i=0}^{\text{end-time} \approx 10s} t_i |e_i|) \quad (19)$$

در معادلات فوق $0v$ در صد فرجهش، e_{ss} خطای حالت دایم، t_s زمان نشست، t_r زمان صعود پاسخ پله سیستم می‌باشد. ضریب β را بر اساس نوع سیستم می‌توان تغییر داد. در این آزمایش β برابر با 1 می‌باشد. e_i خطای پاسخ پله سیستم در i امین نمونه می‌باشد. بردار K همان ضرایب کنترلر PID ($[K_p \ K_i \ K_d]$) است. از آنجا که محاسبات انجام شده به صورت عددی می‌باشد در تابع بهینه ساز انتخاب شده ITAE، زمان پایان را 10s انتخاب شده است. استفاده از Avrege به خاطر ایجاد امکان مقایسه این

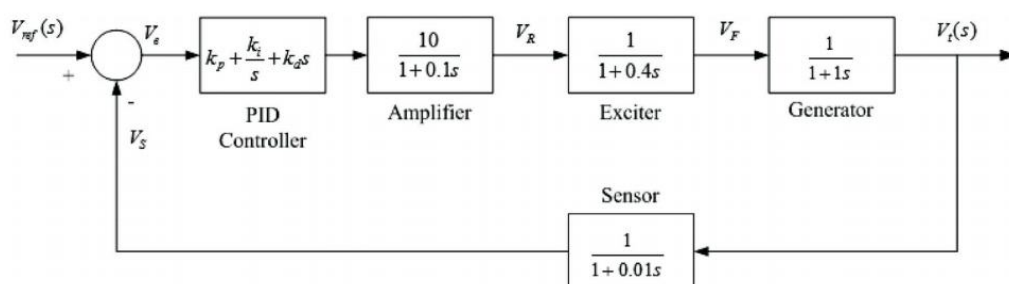
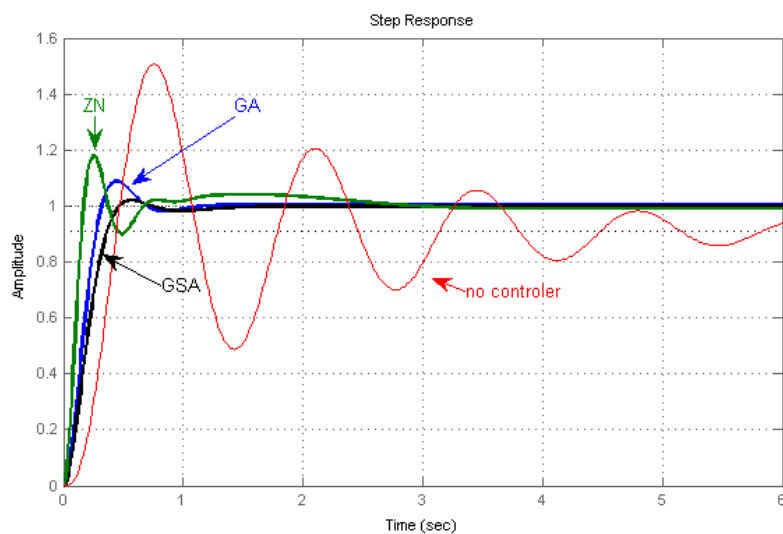


Figure 3. A practical high-order AVR system controlled by a PID controller.

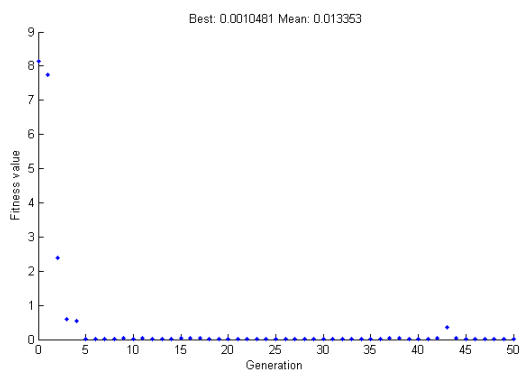
شکل 2. تابع تبدیل سیستم [2]

جدول 2- نتایج به دست آمده از شبیه سازی

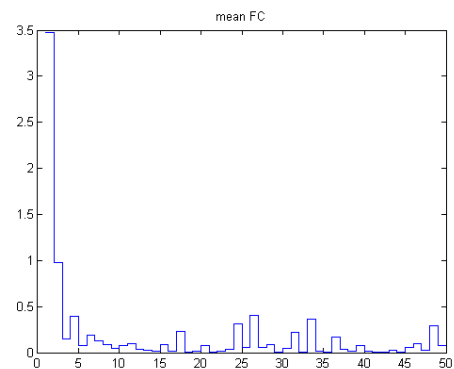
روش	K_p	K_i	K_d	T_r	T_s	M_p	e_{ss}	Avrege (ITAE)	w(K)
No control	1	0	0	0.261	6.990	65.7%	0.0910	0.5241	1.545
ZN	1.012	1.874	0.690	0.115	2.360	18%	0	0.195	0.1832
GA	0.896	0.656	0.291	0.214	0.635	8.08%	0	0.0044	0.0009265
GSA	0.661	0.456	0.218	0.291	0.434	1.97%	0	0.0046	0.0002959



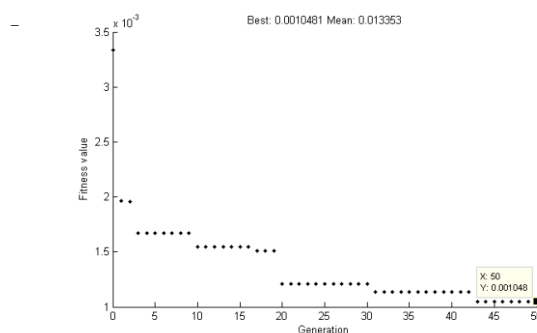
شکل 3- پاسخ پله سیستم با کنترلرهای طراحی شده



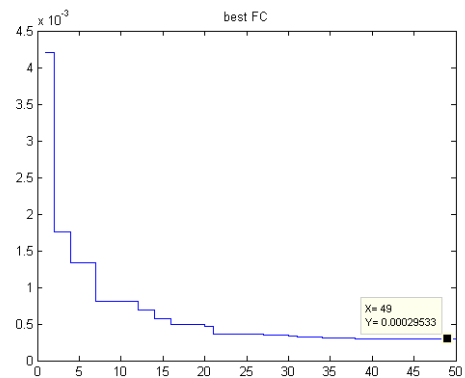
شکل 5- متوسط مقادیر تابع هدف در هر تکرار الگوریتم GA



شکل 4- متوسط مقادیر تابع هدف در هر تکرار الگوریتم GSA



شکل 7- بهترین مقادیر تابع هدف در هر تکرار الگوریتم GA



شکل 6- بهترین مقادیر تابع هدف در هر تکرار الگوریتم GSA

- [1] Esmat Rashedi , Hossein Nezamabadi-pour and Saeid Saryazdi, "GSA: A Gravitational Search Algorithm", Information Sciences 179 (2009) 2232–2248.
- [2] Ching-Chang Wong , Shih-An Li and Hou-Yi Wang," optimal pid controler design for AVR system", Tamkang Journal of Science and Engineering, Vol. 12, No. 3, pp. 259270 (2009).
- [3] E. Atashpaz Gargari & Caro Lucas, “Designing an optimal PID controller using Colonial Competitive Algorithm”, First Iranian Joint Congress on Fuzzy and Intelligent Systems, Mashhad, Iran, 2007